

Gdańsk, 2026
Multimedialne Systemy Interaktywne

Gra edukacyjna do nauki logiki cyfrowej

Alla Krylova 196722
Pavel Khmialeuski 197055

1 Analiza wymagań

1.1 Cel aplikacji

Aplikacja jest edukacyjną grą hybrydową (fizyczne karty + AR), która uczy podstaw logiki cyfrowej. Gracz otrzymuje zadanie zbudowania układu logicznego, który zapali wyjście LED, używając określonego zestawu kart bramek dostępnych na konkretnym poziomie. W tym celu wybiera odpowiednie karty fizyczne (wejścia 0/1, bramki AND, OR, NOT, kartę wyjścia LED) i układa je na stole. Następnie za pomocą urządzenia mobilnego (docelowo z systemem operacyjnym iOS) z aplikacją skanuje obszar gry, rozpoznaje karty i łączy je wirtualnymi przewodami (przeciąganie palcem po ekranie). Aplikacja wizualizuje przepływ sygnału - przewody świecą na zielono przy wartości logicznej 1, na szaro przy 0, a dioda na karcie wyjścia zapala się, gdy cały układ daje wynik 1. Gracz może modyfikować układ (zmieniać wartości wejść przez obrót kart, dokładać/usuwać elementy) i natychmiast obserwować zmiany.

Przykłady podobnych rozwiązań: Logic Gates - Game (Neurotock)

1.2 Grupa docelowa i dystrybucja

Grupa docelowa: studenci kierunków informatycznych i pokrewnych (WETI, elektronika, automatyka), uczniowie szkół średnich o profilu technicznym, hobbyści elektroniki. Sposób dystrybucji: Aplikacja dostępna do pobrania za darmo poprzez:

- TestFlight (do testów i prezentacji)
- Repozytorium GitHub z plikiem .ipa (dla użytkowników z kontem developerskim)
- Docelowo możliwość publikacji w App Store (wymaga konta Apple Developer - 99

USD/rok)

Aspekty prawne: wykorzystywane biblioteki (Vuforia, Unity) są darmowe do celów edukacyjnych/niekomercyjnych. Grafiki i modele 3D będą wykonane samodzielnie lub pochodzić z darmowych źródeł.

1.3 Platforma docelowa

Typ	Minimalne wymagania	Zalecane
System operacyjny	iOS 14.0	iOS 16.0 lub nowszy
Sprzęt	iPhone 6s	iPhone 11 lub nowszy
Kamera	Aparat 12 MP	Aparat 12 MP z OIS
Czujniki	Akcelerometr, żyroskop	Akcelerometr, żyroskop
Łączność	Wi-Fi/dane komórkowe	Wi-Fi

Aplikacja będzie testowana na urządzeniach członków zespołu (modele: iPhone 16 Pro (iOS 26.3), iPhone 15 (iOS 26.3)).

1.4 Środowisko (kontekst) działania

Aplikacja działa lokalnie na urządzeniu mobilnym - nie wymaga stałego połączenia z internetem (poza pobraniem i ewentualnymi aktualizacjami).

Wykorzystywane technologie:

- Unity 6.0 LTS - silnik gry
- Vuforia Engine - rozpoznawanie markerów (obrazów na kartach)
- .NET C# - logika aplikacji

1.5 Wymagania

1.5.1 Funkcjonalne:

- Rozpoznawanie fizycznych kart (wejścia 0/1, bramki AND, OR, NOT, wyjście LED) za pomocą kamery.
- Wyświetlanie wirtualnych 3D modeli nad kartami.
- Ręczne tworzenie połączeń między pinami elementów (przeciąganie palcem).
- Usuwanie połączeń (długi tap na przewód).
- Automatyczne obliczanie wartości logicznych na podstawie typu bramki i stanów wejść.
- Wizualizacja stanu przewodów (kolor zielony = 1, szary = 0).
- Wizualizacja stanu diody LED (świeci/gaśnie).
- Resetowanie wszystkich połączeń.
- Możliwość zapisu i wczytania schematu (opcjonalnie).

1.5.2 Wydajnościowe:

- Płynność animacji: minimum 30 FPS na iPhone'ach od 6s wzwyż.
- Czas detekcji karty < 0,5 s.
- Opóźnienie rysowania przewodów < 0,1 s.

1.5.3 Jakościowe:

- Intuicyjny interfejs - użytkownik powinien domyślić się, jak tworzyć połączenia bez instrukcji.
- Stabilność działania - brak błędów w typowych scenariuszach.
- Czytelność wizualizacji - przewody nie powinny się plątać.

1.5.4 Sprzętowe:

- iPhone z aparatem i żyroskopem (wszystkie modele od iPhone 6s spełniają wymagania).
- Ekran dotykowy min. 4,7 cala (iPhone SE) - interfejs dostosowany do mniejszych ekranów.

1.5.5 Skalowalność:

- Obsługa różnych rozdzielczości ekranu (od iPhone SE do iPhone Pro Max).
- Automatyczne skalowanie interfejsu (Unity Canvas Scaler).
- Możliwość dodania nowych typów bramek (XOR, NAND, NOR) bez przebudowy całej logiki.

1.5.6 Możliwości dalszego rozwoju:

- Dodanie trybu „wyzwania” - zadania do samodzielnego rozwiązania.
- Wersja na iPada (wykorzystanie większego ekranu).
- Wykorzystanie ARKit do lepszego śledzenia kart (bez markerów - opcjonalnie).
- Synchronizacja schematów przez iCloud.

1.5.7 Otwartość systemu:

- Dokumentacja API.

- Licencja open source (MIT).

1.5.8 Niezawodnościowe:

- Walidacja poprawności połączeń (np. brak zwarcia wyjść).
- Zabezpieczenie przed zapętlaniem (przy ewentualnych sprzężeniach zwrotnych).
- Komunikaty błędów w przypadku problemów z kamerą lub oświetleniem.

1.6 Interfejs użytkownika (UI) i sterowanie

Główny widok: Obraz z kamery + wirtualne elementy 3D.

Tworzenie połączeń:

- Krótki tap na wirtualnym pinie wyjściowym -> wejście w tryb łączenia.
- Tap na pinie wejściowym -> utworzenie przewodu.
- Alternatywnie: przeciągnięcie palcem od pinu do pinu.
- Usuwanie połączenia: Długi tap na przewodzie -> przycisk „Usuń”.
- Reset: Przycisk „Resetuj wszystko” w rogu ekranu.

Zmiana wartości wejścia:

- Fizyczny obrót karty na drugą stronę (jeśli zaprojektujemy dwustronne wejścia)
- Lub przycisk w UI (dla uproszczenia)
- Podgląd zadania: Panel boczny z opisem funkcji do zrealizowania.
- Zgodność z iOS HIG (Human Interface Guidelines):
- Gestów tap/długi tap zgodne z systemowymi.
- Przyciski o odpowiedniej wielkości (min. 44x44 pt).

1.7 Opcjonalne rozwiązania (pożądane, ale nieobowiązkowe)

- Dźwięki (np. kliknięcie przy tworzeniu przewodu, „brzęczyk” przy błędzie).
- Animacja przepływu sygnału (płynące światełka).
- Wczytywanie własnych zestawów kart przez użytkownika.
- Wersja demonstracyjna z gotowymi schematami.
- Tryb gry na czas.
- Wsparcie dla ciemnego motywu (iOS 13+).
- Haptics (silnik Taptic Engine) - lekkie wibracje przy tworzeniu połączeń.

1.8 Inne założenia

Architektura: Model MVC (Model - dane logiczne, View - wizualizacja AR, Controller - obsługa zdarzeń). Wykorzystanie wzorca Observer do aktualizacji UI po zmianach w schemacie. Przechowywanie stanu połączeń w liście obiektów Wire. Kompatybilność wsteczna: iOS 14 jako minimalna wersja (ok. 95% aktywnych urządzeń).

1.9 Odporność na awarie i bezpieczeństwo

- Aplikacja nie przechowuje danych wrażliwych.
- W przypadku utraty sygnału z kamery - komunikat z prośbą o przywrócenie obrazu.
- Automatyczne zapisywanie stanu gry (co 30 sekund) - opcjonalnie.

- Zabezpieczenie przed przeciążeniem procesora - ograniczenie liczby przewodów (np. max 30).
- Aplikacja nie wymaga dostępu do internetu - brak ryzyka wycieku danych.

1.10 Niezbędna dokumentacja

Zgodnie z wymogami przedmiotu MSI, w ramach projektu powstaną następujące dokumenty:

- Wybór tematu (WT)
- Analiza wymagań (AW)
- Opis implementacji (OI)
- Instrukcja użytkownika (IU)
- Krótki opis (KO)

1.11 Zakładane ograniczenia aplikacji

- Działa tylko na urządzeniach z systemem iOS.
- Wymaga dobrych warunków oświetleniowych do stabilnego rozpoznawania kart.
- Maksymalna liczba jednocześnie śledzonych kart: 10112 (ograniczenie wydajnościowe).
- Brak obsługi bardzo skomplikowanych układów (powyżej 20 elementów) - ze względu na czytelność na małym ekranie.
- Brak automatycznego rozpoznawania połączeń - wszystkie są tworzone ręcznie (co jest celowym zabiegiem edukacyjnym).
- Publikacja w App Store wymaga konta Apple Developer (opłata 99 USD/rok) - na etapie projektu korzystamy z instalacji bezpośredniej przez Xcode.

2 Projekt systemu

2.1 Architektura systemu

System oparty jest na architekturze hybrydowej łączącej silnik gry Unity z frameworkiem rozszerzonej rzeczywistości Vuforia. Głównym modelem architektonicznym jest Model-View-Controller (MVC), który zapewnia separację logiki biznesowej od warstwy prezentacji, co ułatwia testowanie i dalszy rozwój aplikacji.

2.1.1 Struktura głównych bloków funkcjonalnych

System dzieli się na cztery warstwy:

Warstwa prezentacji (View):

- AR Camera Controller: Zarządza strumieniem wideo i nakładaniem obiektów 3D.
- Canvas Manager: Obsługuje UI 2D (przyciski, panele, komunikaty).
- Wire Renderer: Odpowiada za rysowanie linii (przewodów) między pinami

elementów. Warstwa logiki aplikacji (Controller):

- Input Handler: Interpretuje gesty użytkownika (tap, przeciąganie, długi tap).
- Connection Manager: Zarządza tworzeniem i usuwaniem połączeń oraz walidacją

poprawności schematu.

- **Simulation Engine:** Oblicza wartości logiczne na podstawie topologii układu

(propagacja sygnału). Warstwa danych (Model):

- **Circuit Graph:** Reprezentuje układ jako graf skierowany (węzły - piny, krawędzie - przewody).

- **Gate Models:** Definicje typów bramek (AND, OR, NOT) wraz z ich funkcjami logicznymi.

- **Card Registry:** Baza danych znaczników Vuforia mapujących fizyczne karty na typy elementów. Warstwa integracji sprzętowej:

- **Vuforia Bridge:** Komunikuje się z SDK Vuforia, dostarcza informacje o pozycji i stanie kart.

2.1.2 Diagramy

Diagram przypadków użycia (Use Case):

Diagram klas (uproszczony):

Diagram przepływu danych (Data Flow):

- **Wejście:** Kamera (obraz) -> Vuforia (identyfikacja kart) -> Input Handler

(tapy/przeciągnięcia).

- **Przetwarzanie:** Connection Manager (aktualizacja grafu) -> Simulation Engine

(obliczenia boolowskie).

- **Wyjście:** Simulation Engine -> Wire Renderer (kolory przewodów) -> AR Overlay

(stan diody LED) -> Canvas UI.

2.2 Środowisko tworzenia aplikacji

Element	Wybór	Uzasadnienie
Język programowania	C#	Główny język skryptowy w silniku Unity. Wysoki poziom abstrakcji, bezpieczeństwo typów.
Silnik gry / Środowisko wytworcze	Unity 6.0 LTS	Stabilna wersja LTS (Long Term Support) z długim cyklem wsparcia. Kompatybilna z iOS i najnowszymi SDK firmy Apple.
Framework AR	Vuforia Engine 10.20	Sprawdzony framework do rozpoznawania markerów obrazowych. Wspiera Unity i iOS. Bezpłatny dla celów edukacyjnych.
IDE	Visual Studio 2022 oraz Visual Studio Code for Mac	Visual Studio oferuje zaawansowane wsparcie dla Unity, Visual

		Studio Code jest darmowe i w pełni funkcjonalne.
Szablon projektowy	MVC (Model-View-Controller) + Observer	Zapewnia separację logiki od UI. Wzorzec Observer (np. poprzez zdarzenia C#) pozwala na automatyczną aktualizację wizualizacji po zmianie stanu modelu.
System kontroli wersji	Git (repozytorium GitHub)	Standard w branży. Pozwala na śledzenie zmian i współpracę.

2.3 Projekt najważniejszych struktur danych

2.3.1 Struktura bazy danych

Aplikacja nie korzysta z zewnętrznej bazy danych SQL. Stan gry przechowywany jest w pamięci RAM w postaci obiektów C#. Do zapisu/wczytania schematu (opcjonalnie) wykorzystany zostanie format JSON (serializacja/deserializacja obiektów Circuit i Wire), który będzie zapisywany w pliku w katalogu dokumentów aplikacji (Application.persistentDataPath).

2.3.2 Diagram głównych struktur danych w pamięci

2.3.3 Opis kluczowych klas

- **Circuit:** Klasa kontenerowa. Przechowuje słownik wszystkich komponentów (`Dictionary<string, Component>`) oraz listę połączeń (`List<Wire>`). Odpowiada za inicjalizację propagacji sygnału.
- **Component:** Klasa abstrakcyjna. Posiada unikalne ID, listę pinów wejściowych/wyjściowych oraz metodę abstrakcyjną `Evaluate()`.
- **Wire:** Klasa reprezentująca połączenie. Przechowuje referencje do pinu źródłowego i docelowego. Metoda `Propagate()` przekazuje wartość logiczną z pinu źródłowego do pinu docelowego.

2.4 Projekt interfejsu użytkownika

Interfejs jest projektowany z myślą o wygodzie użytkowania na urządzeniach mobilnych z systemem iOS. Wszystkie elementy interaktywne mają rozmiar minimum 44x44 punkty (pt), zgodnie z wytycznymi iOS HIG.

2.4.1 Główny ekran gry (w trybie AR)

Jest to główny i jedyny widok aplikacji. Obraz z kamery stanowi tło, na które nakładane są elementy UI: Górny pasek:

- Lewy róg: Przycisk „Zadanie” (ikona listy). Rozmiar: 44x44 pt. Po wciśnięciu wysuwa się panel boczny z opisem funkcji do zrealizowania (np. „Zapal diodę, gdy A=1 i B=0”).
- Środek: Nazwa poziomu / trybu (np. „Poziom 1: Podstawy”).
- Prawy róg: Przycisk „Reset” (ikona kosza/odświeżenia). Rozmiar: 44x44 pt. Po naciśnięciu pojawia się alert z potwierdzeniem „Usunąć wszystkie połączenia?”.

Dolny pasek (opcjonalnie):

- Przycisk „Zapisz schemat” (opcjonalnie).
- Przycisk „Wczytaj schemat” (opcjonalnie).

2.4.2 Tworzenie i zarządzanie połączeniami

- Tryb łączenia: po krótkim wciśnięciu na pinie wyjściowym, pin zostaje podświetlony (np. niebieska poświata), a kursor zmienia się na ikonę „plug”. UI wyświetla komunikat wyjaśniający użytkownikowi, co ma zrobić (np. połączyć pin z innym pinem).
- Wizualizacja przewodu: linia 3D łącząca dwa piny. Grubość linii wynosi około 5 punktów w skali ekranu, dla czytelności. Kolor:
 - Zielony (#00FF00): gdy wartość logiczna na przewodzie wynosi 1.
 - Szary (#808080): gdy wartość logiczna na przewodzie wynosi 0.
- Usuwanie przewodu: długie przytrzymanie na przewodzie powoduje wyświetlenie przycisku „Usuń” w miejscu dotknięcia lub podświetlenie przewodu na czerwono i pojawienie się ikony kosza.

2.4.3 Ekrany powiadomień

Alert błędu - modalne okno dialogowe (zgodne z UIAlertController na iOS) wyświetlane w przypadku:

- Próby połączenia wyjścia z wyjściem.
- Próby utworzenia pętli (połączenia, które stworzyłyby cykl w grafie).
- Utraty śledzenia kamery (komunikat: „Nie mogę znaleźć kart. Upewnij się, że są dobrze oświetlone i w polu widzenia.”).

2.5 Wykorzystywane zasoby

Kategoria	Zasób	Przeznaczenie
Biblioteki	Unity Engine (Core, AR, UI)	Podstawowy silnik renderujący, fizyka, obsługa wejścia.
Biblioteki	Vuforia Engine	Rozpoznawanie obrazów (markerów) na kartach, śledzenie pozycji w przestrzeni.
Biblioteki	Newtonsoft.Json (dla Unity)	Serializacja/deserializacja stanu układu do formatu JSON.
Algorytmy	Propagacja sygnału (BFS/Topological Sort)	Algorytm przechodzenia grafu połączeń w celu obliczenia wartości logicznych. Zapewnia stabilność i wykrywanie cykli.
Algorytmy	Raycasting (Unity)	Wykrywanie, który pin został dotknięty przez użytkownika.
Wzorce projektowe	MVC	Separacja odpowiedzialności.
Wzorce projektowe	Observer (C# Events)	Komunikacja między Circuit (Model) a WireRenderer (View).

Wzorce projektowe	Factory Method	Tworzenie różnych typów komponentów (GateFactory).
Wzorce projektowe	Singleton	Dla ConnectionManager i SimulationEngine (jedna instancja na całą aplikację).
Dane multimedialne	Modele 3D (FBX/OBJ)	Proste, stylizowane modele bramek logicznych, wejść, diody LED.
Dane multimedialne	Tekstury (PNG)	Tekstury na karty fizyczne (do wydruku) oraz materiały dla modeli 3D.
Dane multimedialne	Czcionki (TTF/OTF)	Czcionka systemowa San Francisco (iOS) dla zachowania spójności z HIG.

2.6 Realizowane główne funkcje systemu

Funkcja	Sposób wywołania	Parametry wejściowe	Opis / Efekt
Rozpoznanie karty	Automatycznie (klatka kamery)	Obraz z kamery	Vuforia identyfikuje znacznik, tworzy instancję obiektu <i>Component</i> na scenie.
Utworzenie połączenia	Krótkie wciśnięcie na pinie wyjściowym -> krótkie wciśnięcie na pinie wejściowym	Referencje: Pin sourcePin, Pin targetPin	Walidacja poprawności (wyjście->wejście). Dodanie obiektu Wire do Circuit. Wywołanie <code>SimulationEngine.Evaluate()</code> .
Usunięcie połączenia	Długie przytrzymanie na przewodzie -> wybór opcji „Usuń”	Referencja: Wire wireToRemove	Usunięcie obiektu Wire z Circuit. Wywołanie <code>SimulationEngine.Evaluate()</code> .
Zmiana stanu wejścia	Obrót fizycznej karty / Dotknięcie wirtualnego przycisku nad kartą wejścia	Identyfikator: inputComponentId	Zmiana wartości bool w obiekcie InputSource. Wywołanie <code>SimulationEngine.Evaluate()</code> .
Obliczenie wartości logicznych	Automatycznie po każdej zmianie topologii lub stanu wejść	Brak (działa na bieżącym stanie Circuit)	Przejsie grafu od źródeł (InputSource) do wyjść (LedOutput) z wykorzystaniem funkcji logicznych bramek. Aktualizacja pól CurrentValue w każdym Wire.
Wyczyszczenie układu	Naciśnięcie przycisku „Reset”	Brak	Usunięcie wszystkich obiektów Wire z Circuit. Ustawienie

			wszystkich wejść na false. Wywołanie <code>SimulationEngine.Evaluate()</code>
Zapis schematu (opcjonalnie)	Naciśnięcie przycisku „Zapisz”	Ścieżka pliku (do- myślna)	Serializacja obiektów Circuit (bez referencji do obiektów Unity) do JSON. Zapis do pliku.
Wczytanie schematu (opcjonalnie)	Naciśnięcie przycisku „Wczytaj”	Ścieżka pliku (do- myślna)	Odczyt JSON, deserializacja, odtworzenie struktury Circuit i połączeń w scenie.

3 Dokumentacja wersji beta

3.1 Aktualny stan zaawansowania projektu

Projekt znajduje się w fazie zaawansowanego prototypu (wersja beta). Wszystkie kluczowe funkcjonalności wymienione w analizie wymagań zostały zaimplementowane i przetestowane w środowisku docelowym. Poniższa tabela przedstawia zrealizowane zadania.

ID	Nazwa zadania	Status
MSI-5	Projekt fizycznych kart	Zakończone
MSI-7	Podstawowa detekcja kart (Vuforia)	Zakończone
MSI-11	System tworzenia połączeń	Zakończone
MSI-12 .. 15	Menedżer połączeń, wizualizacja przewodów, gesty	Zakończone
MSI-16	Utworzenie podstawowego projektu Unity	Zakończone
MSI-17	Logika obliczeń (AND, OR, NOT)	Zakończone
MSI-21	Interfejs użytkownika (UI)	Zakończone
MSI-22 .. 24	Podstawowe UI, ekran zadania	Zakończone
MSI-26 .. 31	System poziomów, menedżer poziomów, odpowiedzi	Zakończone
MSI-37	Fix kart wejścia (nie rozpoznawały się)	Zakończone
MSI-38	Dodanie dodatkowych kart wejścia	Zakończone
MSI-39	Różne poprawki i stabilizacja	Zakończone

Główne komponenty działające w wersji beta:

- Rozpoznawanie 6 typów fizycznych kart (wejście 0, wejście 1, AND, OR, NOT, wyjście LED).
- Tworzenie i usuwanie połączeń (przez tapnięcie na pinie wyjściowym, a następnie na wejściowym; usuwanie przez długie przytrzymanie na przewodzie).
- Automatyczne obliczanie wartości logicznych z propagacją sygnału przez cały układ.
- Wizualizacja stanu przewodów (zielony = 1, szary = 0) oraz diody LED (świeci/gasną).
- System 6 poziomów o rosnącym stopniu trudności, z listą dozwolonych kart, podpowiedziami i opisem zadania.
- Panel wyboru poziomu.

- Walidacja poprawności układu (użycie wszystkich wymaganych kart, brak dodatkowych niedozwolonych kart).
- Menu hamburger z opcjami: reset wszystkich połączeń, podpowiedź, lista dozwolonych kart.
- Komunikaty błędów dla sytuacji, gdy użytkownik próbuje użyć niedozwolonej karty lub nie użył wszystkich wymaganych.

3.2 Trudności w realizacji

W trakcie implementacji napotkano następujące problemy oraz znaleziono dla nich rozwiązania:

1. **Konflikt Vuforia z AR Foundation** - projekt został początkowo utworzony z szablonu AR Mobile, który zawierał komponenty AR Foundation (XR Origin). Uniemożliwiało to poprawną inicjalizację Vuforia. **Rozwiązanie:** Usunięto wszystkie obiekty i pakiety związane z AR Foundation, pozostawiając wyłącznie AR Camera z Vuforia.
2. **Wykrywanie długiego tapu na przewodzie** - przewody są bardzo cienkie, a LineRenderer nie posiada domyślnie kolizji. **Rozwiązanie:** Dodano dynamiczny BoxCollider dostosowywany do długości i rotacji przewodu, a w obsłudze gestów sprawdzano, czy kolizja pochodzi od przewodu (a nie od pinu).
3. **Walidacja dodatkowych kart** - początkowo sprawdzano wszystkie karty w scenie, w tym nieaktywne (nie śledzone przez Vuforia), co powodowało fałszywe alarmy. **Rozwiązanie:** Dodano sprawdzenie flagi IsTracked (z ObserverBehaviour), aby uwzględniać tylko karty fizycznie widoczne.

3.3 Różnice między założeniami a obecną implementacją (wersja beta)

W implementacji wersji beta wprowadzono następujące różnice względem pierwotnej analizy wymagań.

- **Sposób reprezentacji wartości wejściowych** - zamiast dwustronnych kart fizycznych (0/1 na dwóch stronach) lub zmiany wartości wyjścia poprzez UI zastosowano dwie osobne karty: jedną z symbolem 0, drugą z symbolem 1. Upraszcza to produkcję markerów i nie wymaga odwracania kart.
- **Opcjonalne funkcje** - dźwięki, haptika, animacja przepływu sygnału nie zostały zaimplementowane w tej wersji. Priorytetem była stabilność systemu poziomów i poprawność obliczeń logicznych.
- **Brak komunikatu o słabym oświetleniu** - w obecnej wersji beta użytkownik nie jest ostrzegany o pogorszeniu warunków oświetleniowych (funkcja planowana na finalną wersję).

3.4 Wyniki wstępnych testów (weryfikacja i walidacja)

Testy przeprowadzono w dwóch środowiskach:

- **Środowisko wytwarzania** - Unity Editor (tryb Play Mode z kamerą wirtualną).
- **Środowisko docelowe** - fizyczne urządzenia iPhone 15 (iOS 26.3) i iPhone 16 Pro (iOS 26.3).

3.4.1 Testy weryfikacyjne (w edytorze Unity)

Test	Wynik
Rozpoznawanie markerów wszystkich typów kart	Pozytywny - karty są identyfikowane, a nad nimi wyświetlane są modele 3D.

Tworzenie przewodu między wyjściem a wejściem	Po tapnięciu na pinie wyjściowym (podświetlenie), a następnie na wejściowym - przewód pojawia się.
Usuwanie przewodu przez długie przytrzymanie	Przytrzymanie na przewodzie powoduje jego usunięcie.
Propagacja sygnału: Input 0 -> NOT -> Output	Obliczane poprawnie: NOT(0)=1, przewód między NOT a Output zmienia kolor na zielony, dioda LED zapala się.
Propagacja sygnału: AND z dwoma wejściami 1	AND(1,1)=1, przewód zielony, dioda świeci.
Propagacja sygnału: AND z jednym wejściem odłączonym	Niepodłączone wejście jest traktowane jako 0, więc wynik AND = 0, przewód szary, dioda wyłączona.
Walidacja niedozwolonych kart (extra card)	Próba podłączenia pinu z karty nieznajdującej się na liście dozwolonej dla poziomu - pojawia się popup z informacją, połączenie nie zostaje utworzone.
Walidacja wymaganych kart (wszystkie muszą być użyte)	Jeśli brakuje połączenia do wymaganej karty (np. NOT w poziomie NOT Gate), poziom nie zostaje ukończony, pojawia się odpowiedni komunikat.
System poziomów - przejście do następnego poziomu	Po kliknięciu „Next Level” w popupie sukcesu, ładuje się kolejny poziom.

3.4.2 Testy walidacyjne (na fizycznych urządzeniach)

Przeprowadzono testy na rzeczywistych kartach wydrukowanych na matowym papierze, w różnych warunkach oświetleniowych.

Test	Wynik
Śledzenie kart	Do 10 kart jednocześnie śledzonych stabilnie (wymagane do wszystkich poziomów).
Czas detekcji nowej karty	Średnio 0,3-0,5 s.
Płynność animacji	30-40 FPS na obu urządzeniach, brak odczuwalnych spadków.
Tworzenie przewodów	Gesty tap działają poprawnie, nawet gdy użytkownik trzyma telefon jedną ręką. Podświetlenie pinów ułatwia zrozumienie trybu łączenia.
Usuwanie przewodów	Długie przytrzymanie na przewodzie niezawodne dla przewodów dłuższych niż 5 cm. Dla bardzo krótkich (np. bezpośrednio między sąsiadującymi kartami) czasami trudno trafić - problem do poprawy w finalnej wersji (zwiększenie grubości kolizji).
Oświetlenie	W świetle dziennym i przy biurkowym - rozpoznawanie bardzo dobre. Przy słabym oświetleniu - spadek skuteczności.
Zużycie baterii	Po 30 minutach gry: spadek o ok. 15%.

3.4.3 Zidentyfikowane braki (do poprawy w wersji finalnej)

- Trudność w usuwaniu bardzo krótkich przewodów (zwiększyć rozmiar kolizji dynamicznie w zależności od długości).
- Brak możliwości zmiany wartości wejść przez UI w obecnej wersji.
- Sporadyczne błędy przy próbie połączenia pinu wyjściowego z pinem wyjściowym (obecnie blokowane, ale komunikat nie zawsze jest czytelny).
- Aplikacja nie zawiera jeszcze instrukcji użytkownika (powstanie w ramach dokumentu IU).

3.5 Pozostałe zadania do wykonania

Następujące funkcjonalności wymagają jeszcze ukończenia lub dopracowania przed wydaniem finalnym:

ID	Nazwa zadania	Status
MSI-6	Wykonanie / znalezienie modeli 3D	Do zrobienia (Todo)
MSI-25	System poziomów (dokończenie integracji)	Do zrobienia (Todo)
MSI-32	Zapisywanie postępu	W toku (In Progress)
MSI-33	Testowanie (ogólne)	Do zrobienia (Todo)
MSI-34	Testowanie na rzeczywistych urządzeniach	Do zrobienia (Todo)
MSI-35	Ulepszenia UX (doświadczenie użytkownika)	Do zrobienia (Todo)
MSI-36	Optymalizacja (wydajność)	Do zrobienia (Todo)

3.6 Zrzuty ekranu

- **Rys. 1** - Ekran główny w trybie AR: widok kamery z kartami AND i OR, nad nimi modele 3D z pinami, zielony przewód między AND a OR.
- **Rys. 2** - Panel wyboru poziomów: lista poziomów (NOT, AND, OR, AND-OR Chain, NOT-AND, AND-AND-OR).
- **Rys. 3** - Popup informacyjny przed rozpoczęciem poziomu: tytuł poziomu i lista dozwolonych kart.
- **Rys. 4** - Popup błędu „Extra Cards Detected” - wyświetlany gdy użytkownik próbuje użyć karty AND w poziomie NOT Gate.
- **Rys. 5** - Popup sukcesu po ukończeniu poziomu: komunikat z przyciskiem „Next Level”.
- **Rys. 6** - Menu hamburger (rozwinięte) z opcjami: Reset Wires, Hint, Allowed Cards.